

AgriShield AI

Crop and Leaf Scan Upgrade Report

Implementation summary - 16 September 2026

This report documents the new safety-first image pipeline added to the existing AgriShield application. The system now validates the image, identifies the crop using a separately trained model, and only then runs disease prediction.

New processing flow

Upload image -> Leaf validation -> Crop identification -> Disease model -> Result

Non-leaf images, corrupt files, unsupported files, dark images, blurry images, and uncertain crop predictions are rejected before disease prediction. The backend never trusts the crop selected in the browser as the authoritative crop.

What was added

File	Purpose
backend/crop_classifier.py	Loads the optional crop model, applies the 70% confidence threshold, and returns safe unavailable/uncertain errors.
training/train_crop_classifier.py	Trains a MobileNetV2 crop classifier and saves model, labels, and validation metrics.
dataset/crop_leaf_identification/DATASET.md	Documents the real labelled dataset folder structure.
backend/app.py	Runs crop identification after leaf validation and before disease inference; exposes model status.
frontend/app.js	Shows automatic crop identification and confidence in the existing scanner result UI.
backend/test_technical_policy.py	Verifies that the system does not guess a crop when classifier weights are missing.
README.md and training/README.md	Documents the new workflow, training command, and safe limitations.

Successful response

```
{"crop_identification": {"crop": "rice", "confidence": 0.94, "accepted": true, "threshold": 0.70}}
```

The frontend displays: Leaf detected: Yes, Crop identified: Rice, and the crop confidence percentage.

How to train the crop model

1. Add real, licensed and labelled leaf photographs:

```
dataset/crop_leaf_identification/rice/  
dataset/crop_leaf_identification/wheat/  
dataset/crop_leaf_identification/maize/  
dataset/crop_leaf_identification/tomato/
```

2. Add more crop folders only when you have enough representative images for each class.
3. Run the training script from the project root:

```
python training/train_crop_classifier.py --data dataset/crop_leaf_identification  
--epochs 15
```

4. The script writes:

```
models/crop_leaf_classifier.keras  
models/crop_leaf_classifier.labels.json  
models/crop_leaf_classifier.metrics.json
```

Do not claim production accuracy from the training split alone. Keep independent field photographs for evaluation, and record the dataset source, license, class counts, and capture conditions.

Current limitation

The application code is ready, but no crop-classifier weights were fabricated or bundled. Until real weights and labels are installed, the API safely returns a model-unavailable response. The existing disease model currently supports rice only; other crops need their own validated disease models before disease results can be shown.

Verification completed

Check	Result
Backend test suite	11 tests passed
Python compilation	Passed for new and modified Python files
Frontend syntax	Passed with Node.js syntax check
Editor diagnostics	No errors reported in changed files
Missing crop model behavior	Rejected safely; no guessed crop or disease result

Safety principle

When the system is uncertain, it rejects the scan and asks for a clearer leaf image or plant-doctor consultation. It does not invent a crop, disease name, or confidence score.